

ChurnMux — System Architecture Specification

Explainable Customer Churn Prediction Engine with SHAP Force-Plots & Counterfactual Lab

Problem Statement: Problem 16	Tech Stack: LightGBM + SHAP + Flask + React	Version: 1.0.0 (Production)
Frontend: Vercel (Vite SPA)	Backend: Render (Gunicorn WSGI)	ML Validation: 80/20 Stratified Holdout

1. Executive Overview & Problem Statement

Traditional churn prediction algorithms operate as black-boxes: they alert businesses that an account is at risk, but fail to provide actionable causality. **ChurnMux** fulfills **Problem Statement 16** by establishing an end-to-end explainable churn intelligence platform. It unifies high-performance gradient boosted decision trees with Shapley Additive Explanations (SHAP) and a real-time counterfactual simulation engine.

Core Deliverables:

- **Macro Explainability:** Portfolio-wide global feature impact ranking via mean absolute SHAP values.
- **Micro/Local Explainability:** Single-account SVG force-plots decomposing exact positive (churn) and negative (retention) forces.
- **Counterfactual Simulation:** Live 'What-If' lab allowing revenue teams to adjust parameters and witness instant risk reduction.
- **Strict Out-of-Sample Integrity:** Complete holdout separation ensuring zero in-sample evaluation leakage.

2. System Topology & Data Flow

The system is organized into a modular three-tier architecture: the **ML & Explainability Core**, the **Flask REST API**, and the **React 18 Dashboard**.

Tier / Layer	Technology	Responsibilities & Interfaces
Frontend Client	React 18, Vite, Custom SVG, CSS Tokens	Renders Executive KPI Bar, Account Portfolio Table, Global SHAP Bar Chart, SVG Force-Plots, and What-If Lab controls.
Backend API	Flask 3.0, Gunicorn, CORS	Exposes RESTful endpoints (/api/accounts, /api/shap/local, /api/simulate) with sub-50ms inference latency.
ML & SHAP Core	LightGBM 4.7, SHAP 0.51, scikit-learn	Trained on 80% train split; pre-computes TreeExplainer log-odds attributions and stores serialized artifacts.

3. Machine Learning Methodology & Metrics

Data Sanitization: 12 columns were dropped from raw data: 3 target leakage fields (*Churn Reason*, *Churn Score*, *Churn Label*), 3 zero-variance constants (*Count*, *Country*, *State*), and 6 high-cardinality/redundant geographic & ID columns (*CustomerID*, *Lat Long*, *City*, *Zip Code*, *Latitude*, *Longitude*). 11 whitespace records in *Total Charges* were imputed with 0.0 for new subscribers.

Evaluation Metric	Score	Methodological Significance
Held-Out Test ROC-AUC	0.850	Strong discrimination on unseen 20% test cohort (1,409 accounts).
Held-Out Test PR-AUC	0.666	Robust precision-recall area on imbalanced target (26.5% base rate).
Test Recall (Catch Rate)	74.4%	Successfully identifies ~3 out of every 4 departing customers.

5-Fold CV Mean ROC-AUC	0.859 ± 0.012	Cross-validation confirms generalization stability across folds.
------------------------	---------------	--

Mathematical Foundation of SHAP Force-Plots:

For an account with feature vector x , the model prediction $f(x)$ decomposes into the expected value $E[f]$ plus the sum of Shapley values: $f(x) = E[f] + \sum \phi_i$. Positive values ($\phi_i > 0$) represent red forces driving churn, while negative values ($\phi_i < 0$) represent blue forces defending retention.

4. Backend REST API Architecture & Contracts

The backend is structured around a service-oriented Flask design pattern. All model artifacts (*model.joblib*, *explainer.joblib*, *accounts_cache.json*) are loaded into memory at startup for zero-disk-latency inference.

Endpoint	Method	Description	Sample Query / Payload
/api/health	GET	Health check & model metrics.	None
/api/accounts/stats	GET	Macro totals (ARR at risk, Churn signal).	None
/api/accounts	GET	Paginated accounts with risk filters.	?risk_level=CRITICAL&page;=1&limit;=10
/api/accounts/:id	GET	Raw features for single account.	ACC-2222
/api/shap/global	GET	Mean absolute SHAP feature ranking.	None
/api/shap/local/:id	GET	Force decomposition vector for account.	ACC-2222
/api/simulate	POST	Counterfactual What-If simulation.	{"base_account_id":"ACC-2222","overrides":{"contract":"Two year"}}

5. Frontend Architecture & Component Hierarchy

Built with React 18 and Vite, the dashboard provides a high-density, mission-control user experience organized into 5 functional panels:

- **Header & KPI Bar:** Live indicators for API status (latency in ms), Test ROC-AUC (0.850), Total Monitored Accounts (1,409), Portfolio Churn Signal (38.7%), and Critical Revenue at Risk (\$50.7K MRR).
- **Panel 01 (Account Portfolio Explorer):** Real-time filterable grid allowing instant triage by risk level (CRITICAL ≥ 70%, ELEVATED 35-70%, LOW < 35%) with debounced search.
- **Panel 02 (Global Feature Impact):** Horizontal ranked bar chart illustrating macro feature importance across all accounts.
- **Panel 03 (Account Telemetry):** Detailed individual customer view displaying contract terms, tenure, MRR, payment method, and service subscriptions.
- **Panel 04 (SHAP Force Decomposition):** Custom SVG renderer showing the baseline probability anchor (26.5%), red push blocks (e.g. Month-to-month contract +0.988), and blue anchor blocks, pointing to the final predicted score.
- **Panel 05 (What-If Counterfactual Lab):** Interactive controls (contract dropdown, monthly fee slider, service toggles) connected to POST /api/simulate for real-time risk rebalancing.

6. Production Cloud Deployment Topology

Deployment Layer	Cloud Platform	Configuration Details
Frontend (React UI)	Vercel	Root Directory: Client Build: npm run build Output: dist Env: VITE_API_URL points to Render backend.
Backend + Model	Render	Web Service Build: pip install -r requirements.txt Start: gunicorn "Server.app:create_app()" Dynamic \$PORT binding & CORS.